

DBMS – SELLER AND INVENTORY MANAGEMENT

Project Title: Nykaa – Beauty, Skincare, Haircare and Fashion E-Commerce Management System

Skill Task: DBMS

Week: 3

1. Database Design

1.1 Seller Table

Attribute	Data Type	Constraint
Seller ID	INT	Primary Key
Seller Name	VARCHAR2(100)	NOT NULL
Email	VARCHAR2(100)	NOT NULL, UNIQUE
Phone	VARCHAR2(15)	NOT NULL, UNIQUE
Address	VARCHAR2(200)	NOT NULL

1.2 Product Table

The Product table is required so that Inventory can reference a real Product_ID. Create it here only if it was not already created in a previous week.

Attribute	Data Type	Constraint
Product ID	INT	Primary Key
Product Name	VARCHAR2(100)	NOT NULL
Brand	VARCHAR2(50)	–
Category	VARCHAR2(50)	NOT NULL
Price	NUMBER(10,2)	NOT NULL
Stock	INT	NOT NULL
Description	VARCHAR2(255)	–
Seller ID	INT	Foreign Key

1.3 Inventory Table

Attribute	Data Type	Constraint
Inventory ID	INT	Primary Key
Product ID	INT	Foreign Key, UNIQUE
Quantity Available	INT	NOT NULL
Last Updated	DATE	NOT NULL

Note: the entity analysis lists a Stock attribute on Product and a separate Quantity_Available attribute on Inventory. Both are kept here exactly as given, with the same value maintained in each; Section 9 (Inventory Update) updates both together so the two stay in sync.

2. Table Relationships

Parent Entity	Relationship	Child Entity	Explanation
Seller	One-to-Many	Product	One seller can sell many products.
Product	One-to-One	Inventory	One product has one inventory record.

3. Table Creation

3.1 Create Seller Table

```
SQL> CREATE TABLE Seller (  
2     Seller_ID    INT PRIMARY KEY,  
3     Seller_Name  VARCHAR2(100) NOT NULL,  
4     Email        VARCHAR2(100) NOT NULL UNIQUE,  
5     Phone        VARCHAR2(15)  NOT NULL UNIQUE,  
6     Address      VARCHAR2(200) NOT NULL  
7 );
```

Table created.

3.2 Create Product Table

```
SQL> CREATE TABLE Product (  
2     Product_ID   INT PRIMARY KEY,  
3     Product_Name VARCHAR2(100) NOT NULL,  
4     Brand        VARCHAR2(50),  
5     Category     VARCHAR2(50) NOT NULL,  
6     Price        NUMBER(10,2) NOT NULL,  
7     Stock        INT NOT NULL,  
8     Description  VARCHAR2(255),  
9     Seller_ID    INT NOT NULL,  
10    FOREIGN KEY (Seller_ID) REFERENCES Seller(Seller_ID)  
11 );
```

Table created.

3.3 Create Inventory Table

```
SQL> CREATE TABLE Inventory (  
2     Inventory_ID INT PRIMARY KEY,  
3     Product_ID   INT NOT NULL UNIQUE,  
4     Quantity_Available INT NOT NULL,  
5     Last_Updated DATE NOT NULL,  
6     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
7 );
```

Table created.

Note: Product_ID is declared UNIQUE in the Inventory table so that the Product → Inventory relationship stays One-to-One, as defined in the entity design.

4. Seller Data Insertion

```
SQL> INSERT INTO Seller VALUES (501, 'Glow Essentials Pvt Ltd', 'glowessentials@gmail.com', '9000000001',  
'Mumbai');
```

1 row created.

```
SQL> INSERT INTO Seller VALUES (502, 'Herbal Roots Cosmetics', 'herbalroots@gmail.com', '9000000002',  
'Delhi');
```

1 row created.

```
SQL> INSERT INTO Seller VALUES (503, 'Radiant Skin Co', 'radiantskin@gmail.com', '9000000003', 'Bangalore');
```

1 row created.

```
SQL> INSERT INTO Seller VALUES (504, 'Chic Fashion House', 'chicfashion@gmail.com', '9000000004',  
'Hyderabad');
```

1 row created.

```
SQL> INSERT INTO Seller VALUES (505, 'Pure Naturals Store', 'purenaturals@gmail.com', '9000000005',  
'Chennai');
```

1 row created.

5. Product Data Insertion

```
SQL> INSERT INTO Product VALUES (801, 'Matte Liquid Lipstick', 'Glow Essentials', 'Makeup',
  2      499, 60, 'Long-lasting matte finish lipstick', 501);

1 row created.

SQL> INSERT INTO Product VALUES (802, 'Herbal Face Wash', 'Herbal Roots', 'Skincare',
  2      250, 80, 'Neem and tulsi based face wash', 502);

1 row created.

SQL> INSERT INTO Product VALUES (803, 'Argan Hair Serum', 'Radiant Skin Co', 'Haircare',
  2      650, 0, 'Nourishing hair serum with argan oil', 503);

1 row created.

SQL> INSERT INTO Product VALUES (804, 'Floral Eau de Parfum', 'Chic Fashion House', 'Fragrance',
  2      1800, 15, 'Long-lasting floral fragrance', 504);

1 row created.

SQL> INSERT INTO Product VALUES (805, 'Cotton Printed Scarf', 'Pure Naturals Store', 'Fashion',
  2      350, 25, 'Lightweight cotton printed scarf', 505);

1 row created.
```

6. Inventory Data Insertion

```
SQL> INSERT INTO Inventory VALUES (901, 801, 60, '01-SEP-2026');

1 row created.

SQL> INSERT INTO Inventory VALUES (902, 802, 80, '01-SEP-2026');

1 row created.

SQL> INSERT INTO Inventory VALUES (903, 803, 0, '02-SEP-2026');

1 row created.

SQL> INSERT INTO Inventory VALUES (904, 804, 15, '02-SEP-2026');

1 row created.

SQL> INSERT INTO Inventory VALUES (905, 805, 25, '03-SEP-2026');

1 row created.
```

7. Seller and Product Information

7.1 Display Seller and Product Details

This query combines Seller and Product information.

```
SQL> SELECT
  2     s.Seller_Name,
  3     p.Product_Name,
  4     p.Category,
  5     p.Price,
  6     p.Stock
  7 FROM Seller s
  8 JOIN Product p
  9 ON s.Seller_ID = p.Seller_ID
 10 ORDER BY s.Seller_Name;
```

SELLER_NAME	PRODUCT_NAME	CATEGORY	PRICE	STOCK
Chic Fashion House	Floral Eau de Parfum	Fragrance	1800	15
Glow Essentials Pvt Ltd	Matte Liquid Lipstick	Makeup	499	60
Herbal Roots Cosmetics	Herbal Face Wash	Skincare	250	80
Pure Naturals Store	Cotton Printed Scarf	Fashion	350	25
Radiant Skin Co	Argan Hair Serum	Haircare	650	0

5 rows selected.

7.2 Display Product and Inventory Details

This query combines Product and Inventory information.

```
SQL> SELECT
2     p.Product_ID,
3     p.Product_Name,
4     i.Quantity_Available,
5     i.Last_Updated
6 FROM Product p
7 JOIN Inventory i
8 ON p.Product_ID = i.Product_ID
9 ORDER BY p.Product_ID;
```

PRODUCT_ID	PRODUCT_NAME	QUANTITY_AVAILABLE	LAST_UPDA
801	Matte Liquid Lipstick	60	01-SEP-26
802	Herbal Face Wash	80	01-SEP-26
803	Argan Hair Serum	0	02-SEP-26
804	Floral Eau de Parfum	15	02-SEP-26
805	Cotton Printed Scarf	25	03-SEP-26

5 rows selected.

8. Inventory Status

8.1 Display Available Products

```
SQL> SELECT
2     p.Product_ID,
3     p.Product_Name,
4     i.Quantity_Available,
5     'Available' AS Stock_Status
6 FROM Product p
7 JOIN Inventory i
8 ON p.Product_ID = i.Product_ID
9 WHERE i.Quantity_Available > 0;
```

PRODUCT_ID	PRODUCT_NAME	QUANTITY_AVAILABLE	STOCK_STATUS
801	Matte Liquid Lipstick	60	Available
802	Herbal Face Wash	80	Available
804	Floral Eau de Parfum	15	Available
805	Cotton Printed Scarf	25	Available

4 rows selected.

8.2 Display Unavailable Products

```
SQL> SELECT
2     p.Product_ID,
3     p.Product_Name,
4     i.Quantity_Available,
5     'Unavailable' AS Stock_Status
6 FROM Product p
7 JOIN Inventory i
```

```

8 ON p.Product_ID = i.Product_ID
9 WHERE i.Quantity_Available = 0;

```

PRODUCT_ID	PRODUCT_NAME	QUANTITY_AVAILABLE	STOCK_STATUS
803	Argan Hair Serum	0	Unavailable

1 row selected.

9. Inventory Update

Demonstrate updating stock information.

Note: Product ID 803 (Argan Hair Serum) is intentionally left at zero stock so the zero-stock report in Section 10.4 still has a row to show. The restock below is applied to Product_ID 805 (Cotton Printed Scarf) instead. Stock on the Product table and Quantity_Available on the Inventory table are updated together to keep both in sync.

```

SQL> SELECT Product_ID, Product_Name, Quantity_Available
2 FROM Product p
3 JOIN Inventory i ON p.Product_ID = i.Product_ID
4 WHERE i.Quantity_Available = 0;

```

PRODUCT_ID	PRODUCT_NAME	QUANTITY_AVAILABLE
803	Argan Hair Serum	0

1 row selected.

```

SQL> UPDATE Inventory
2 SET Quantity_Available = 40,
3     Last_Updated = '05-SEP-2026'
4 WHERE Product_ID = 805;

```

1 row updated.

```

SQL> UPDATE Product
2 SET Stock = 40
3 WHERE Product_ID = 805;

```

1 row updated.

```

SQL> SELECT p.Product_ID, p.Product_Name, i.Quantity_Available
2 FROM Product p
3 JOIN Inventory i ON p.Product_ID = i.Product_ID
4 WHERE p.Product_ID = 805;

```

PRODUCT_ID	PRODUCT_NAME	QUANTITY_AVAILABLE
805	Cotton Printed Scarf	40

1 row selected.

10. Inventory Status Reports

10.1 Complete Inventory Report

```

SQL> SELECT
2     s.Seller_Name,
3     p.Product_Name,
4     p.Category,
5     i.Quantity_Available,
6     i.Last_Updated
7 FROM Inventory i
8 JOIN Product p
9 ON i.Product_ID = p.Product_ID
10 JOIN Seller s

```

```

11 ON p.Seller_ID = s.Seller_ID
12 ORDER BY i.Inventory_ID;

```

SELLER_NAME	PRODUCT_NAME	CATEGORY	QUANTITY_AVAILABLE	LAST_UPDA
Glow Essentials Pvt Ltd	Matte Liquid Lipstick	Makeup	60	01-SEP-26
Herbal Roots Cosmetics	Herbal Face Wash	Skincare	80	01-SEP-26
Radiant Skin Co	Argan Hair Serum	Haircare	0	02-SEP-26
Chic Fashion House	Floral Eau de Parfum	Fragrance	15	02-SEP-26
Pure Naturals Store	Cotton Printed Scarf	Fashion	40	05-SEP-26

5 rows selected.

10.2 Available and Unavailable Product Count

```

SQL> SELECT
2     CASE WHEN Quantity_Available > 0 THEN 'Available'
3         ELSE 'Unavailable' END AS Stock_Status,
4     COUNT(*) AS Total_Products
5 FROM Inventory
6 GROUP BY CASE WHEN Quantity_Available > 0 THEN 'Available'
7         ELSE 'Unavailable' END;

```

STOCK_STATUS	TOTAL_PRODUCTS
Available	4
Unavailable	1

2 rows selected.

10.3 Total Stock by Seller

```

SQL> SELECT
2     s.Seller_Name,
3     SUM(i.Quantity_Available) AS Total_Stock
4 FROM Seller s
5 JOIN Product p
6 ON s.Seller_ID = p.Seller_ID
7 JOIN Inventory i
8 ON p.Product_ID = i.Product_ID
9 GROUP BY s.Seller_Name
10 ORDER BY s.Seller_Name;

```

SELLER_NAME	TOTAL_STOCK
Chic Fashion House	15
Glow Essentials Pvt Ltd	60
Herbal Roots Cosmetics	80
Pure Naturals Store	40
Radiant Skin Co	0

5 rows selected.

10.4 Products with Zero Stock

This is particularly useful for demonstrating unavailable products.

```

SQL> SELECT
2     p.Product_ID,
3     p.Product_Name,
4     s.Seller_Name,
5     i.Quantity_Available
6 FROM Product p
7 JOIN Inventory i
8 ON p.Product_ID = i.Product_ID
9 JOIN Seller s
10 ON p.Seller_ID = s.Seller_ID
11 WHERE i.Quantity_Available = 0;

```

PRODUCT_ID	PRODUCT_NAME	SELLER_NAME	QUANTITY_AVAILABLE
803	Argan Hair Serum	Radiant Skin Co	0

1 row selected.

10.5 Product Count by Category

```
SQL> SELECT
2     Category,
3     COUNT(*) AS Total_Products
4 FROM Product
5 GROUP BY Category
6 ORDER BY Category;
```

CATEGORY	TOTAL_PRODUCTS
Fashion	1
Fragrance	1
Haircare	1
Makeup	1
Skincare	1

5 rows selected.

11. Conclusion

The Seller and Inventory modules of the Nykaa E-Commerce Management System were created, populated with sample data, and queried successfully. The One-to-Many relationship between Seller and Product and the One-to-One relationship between Product and Inventory were enforced using primary key, unique and foreign key constraints. Join queries, aggregate reports and a stock update confirm that the design maintains data integrity and supports the reporting needs of the marketplace.